Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

1. Objective

The objective of this experiment is to study the evolution of deep convolutional neural network architectures – LeNet-5, AlexNet, VGG16, GoogleNet and ResNet – and to understand, in practice, how transfer learning allows a network pretrained on a large dataset such as ImageNet to be repurposed for a new classification task. This is achieved by loading a pretrained ResNet50 backbone, replacing its original classification head with a new one suited to the ten classes of CIFAR-10, training this head while the convolutional base remains frozen, subsequently fine-tuning the last convolutional block of the backbone at a reduced learning rate, and finally evaluating the resulting model using standard classification metrics – Accuracy, Precision, Recall, F1-score, the Confusion Matrix and the Classification Report – so that the benefit of fine-tuning over a purely frozen-backbone approach can be measured directly.

2. Learning Outcomes

Students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features directly from raw image data, removing the need for handcrafted feature extraction that dominated earlier computer vision pipelines. The evolution of CNN architectures over the past three decades has largely been driven by two competing goals: improving classification accuracy on increasingly difficult benchmarks such as ImageNet, and doing so without a proportional explosion in computational cost or training difficulty. The progression began with LeNet-5, a comparatively shallow network designed for digit recognition, and evolved through AlexNet, VGGNet, GoogleNet and ResNet, each of which introduced a distinct architectural innovation – ReLU activations and GPU training, uniform small convolution filters, multi-branch inception modules, and residual skip connections respectively – that addressed a specific limitation of its predecessors and pushed the achievable depth and accuracy of CNNs substantially further.

4. Background Theory – Evolution of CNN Architectures

4.1 LeNet-5

LeNet-5, proposed by Yann LeCun in 1998, was the first practical convolutional neural network and was designed for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and a small stack of fully connected layers, and it demonstrated for the first time that a network could learn useful image features directly from pixel data rather than relying on manually engineered features. With only around 60 thousand parameters, LeNet-5 trains quickly and remains well suited to constrained tasks such as optical character recognition, though its shallow depth limits how well it generalizes to more complex, natural images such as those in CIFAR-10.

4.2 AlexNet

AlexNet won the 2012 ImageNet Challenge by a wide margin, reducing the top-5 classification error substantially relative to prior approaches, and is widely credited with reigniting mainstream interest in deep learning. Its major innovations were the use of the ReLU activation function in place of the saturating Sigmoid or Tanh units used previously, Dropout regularization to combat overfitting in its large fully connected layers, and, critically, an implementation split across GPUs that made training a network of roughly 61 million parameters computationally feasible. Data augmentation was also used extensively to artificially expand the effective size of the training set.

4.3 VGG16

VGG16 demonstrated that stacking many small 3×3 convolution filters, rather than using fewer, larger filters as in AlexNet, could outperform larger-filter architectures while increasing overall network depth to 16 weight layers. The uniformity of its 3×3 filters throughout the network gives VGG16 a simple, highly regular architecture that is easy to reason about and extend, and its convolutional layers remain a popular choice as a feature extractor for transfer learning, although at 138 million parameters it is considerably more expensive, both in memory and in computation, than the architectures that followed it.

4.4 GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs several convolution operations of different filter sizes in parallel rather than committing to a single kernel size. Specifically, 1×1, 3×3 and 5×5 convolutions and a max pooling branch are applied simultaneously to the same input, and their outputs are concatenated along the channel dimension to yield a multi-scale feature representation at every stage of the network. Crucially, 1×1 convolutions are applied before the larger 3×3 and 5×5 branches purely to reduce the number of input channels first, which sharply reduces both the parameter count and the computational cost of the subsequent larger convolutions. This design allowed GoogleNet to reach a depth of 22 layers while requiring only around 6.8 million parameters, substantially fewer than either AlexNet or VGG16, while achieving better classification performance than both.

4.5 ResNet

Increasing the depth of a CNN indiscriminately tends to cause the **vanishing gradient problem**, in which gradients computed during backpropagation shrink as they are propagated back through many layers, making the earliest layers of a very deep network difficult or impossible to

train effectively. ResNet, proposed by He *et al.* in 2015, solves this issue using **skip (residual) connections**, which allow the input to a block of layers to be added directly to that block's output, giving gradients an unimpeded path to flow backward through the network regardless of its depth. Rather than requiring a stack of layers to learn the full underlying mapping $H(x)$ directly, ResNet reformulates the problem so that the stack instead learns the residual

$$F(x) = H(x) - x$$

so that the block's actual output is recovered as

$$Output = F(x) + x$$

where $F(x)$ is the residual mapping learned by the convolutional layers within the block and x is the identity mapping carried forward by the skip connection. This reformulation makes it considerably easier for a block to learn an identity mapping when that is the optimal behaviour, since the residual branch need only learn to output zero, rather than having to learn the identity function exactly through a stack of nonlinear layers.

$$\text{Input} \rightarrow \text{Conv} \rightarrow \text{Conv} \rightarrow (+) \rightarrow \text{Output}$$

(Residual Block: the skip connection carries the Input directly to the addition node)

The advantages of this design are that it eliminates vanishing gradients even in very deep networks, enables architectures of 50, 101 or more layers to be trained successfully, leads to faster convergence during training, and achieves higher ImageNet accuracy than any of the architectures preceding it. ResNet50, the 50-layer variant used later in this experiment, uses roughly 25.6 million parameters – fewer than either AlexNet or VGG16 despite being substantially deeper, which is a direct consequence of its efficient bottleneck block design.

4.6 Dilated Convolution

Dilated convolution, also called atrous convolution, increases the effective receptive field of a convolution kernel without increasing the number of trainable parameters, by inserting gaps between the kernel's individual weights. The spacing between kernel elements is controlled by the dilation rate D: for a standard convolution $D = 1$, so the kernel weights are contiguous, whereas for a dilated convolution $D > 1$ and zeros are effectively inserted between weights, for example transforming a compact 3×3 kernel such as

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

so that the kernel now spans a larger region of the input while still applying the same number of multiplications. This is particularly useful in tasks such as semantic segmentation, medical image analysis, satellite imagery interpretation and object localization, where a large receptive field is needed to capture context without discarding spatial resolution through pooling.

4.7 Transpose Convolution

Transpose convolution increases the spatial dimensions of a feature map rather than reducing them. Whereas ordinary pooling operations discard spatial information to shrink a feature map, transpose convolution performs a learnable form of upsampling, in which the network itself learns how to expand a low-resolution feature map back to a higher resolution. This operation

underlies applications such as image super-resolution, autoencoder decoders, Generative Adversarial Networks, general image generation and the decoder pathway of semantic segmentation networks, all of which require projecting a compressed representation back into a full-resolution image or feature map.

5. Comparison of Architectures

Architecture	**Depth**	**Parameters**	**Main Contribution**
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

This table makes clear that depth alone is not what determines a network's parameter count: GoogleNet, despite being the second-deepest network in the table at 22 layers, has by far the fewest parameters of the group, a direct consequence of its Inception module's use of 1×1 convolutions to reduce channel counts before more expensive operations. ResNet50, although the deepest architecture listed at 50 layers, requires fewer parameters than both AlexNet and VGG16, illustrating that the residual bottleneck design used throughout ResNet is considerably more parameter-efficient per layer than either AlexNet's large fully connected layers or VGG16's uniform stack of full-width 3×3 convolutions.

6. Transfer Learning Explanation

Transfer learning utilizes a CNN that has already been pretrained on a large dataset such as ImageNet, which contains over a million labelled images across a thousand categories, so that the general-purpose visual features – edges, textures, shapes and object parts – it has already learned can be reused on a new, smaller dataset rather than being learned again from scratch. Instead of training an entire network from randomly initialized weights, the standard transfer learning workflow proceeds as follows: first, a pretrained model is loaded together with its ImageNet weights; its original classification layer, which was designed for 1,000 ImageNet categories, is then removed; the remaining convolutional base is frozen so that its pretrained weights are not disturbed during the earliest phase of training; new Dense layers suited to the target task are added on top of the frozen base; this new classification head is trained while the base remains frozen; and finally, selected convolutional layers – typically the last block, which encodes the most task-specific features – are unfrozen and fine-tuned at a reduced learning rate to adapt the pretrained features more closely to the new dataset. This workflow is illustrated conceptually as

ImageNet Pretrained Model → Remove Classifier → Freeze Layers → Add Dense Layers → Fine Tune → Prediction

The advantages of this approach are faster convergence, since the network does not need to relearn low-level features from scratch; higher accuracy on small datasets, since the pretrained weights already encode a rich, general-purpose visual representation; reduced training time overall; and lower computational cost compared with training a comparably deep network entirely from random initialization.

7. Dataset Description

The experiment uses the **CIFAR-10 Dataset**, loaded from the Hugging Face Hub via the `datasets` library rather than `torchvision.datasets.CIFAR10`, so that each example is exposed as a dictionary containing a PIL image under the `img` key and an integer label under the `label` key.

Property	**Value**
Dataset Name	CIFAR-10
Source	Hugging Face Hub, `uoft-cs/cifar10`
Training Images	50,000
Testing Images	10,000
Original Image Size	$32 \times 32 \times 3$ (RGB)
Resized Image Size	$224 \times 224 \times 3$ (RGB)
Target Classes	10
Learning Type	Supervised Learning
Problem Type	Multi-class Image Classification

Target Classes

Class Label	**Description**
0	airplane
1	automobile
2	bird
3	cat
4	deer
5	dog
6	frog
7	horse
8	ship
9	truck

Since the ResNet50 backbone used in this experiment expects the 224×224 input resolution that ImageNet models were originally trained on, every 32×32 CIFAR-10 image was resized to 224×224 as part of the transform pipeline, in addition to being normalized using the standard ImageNet channel-wise mean and standard deviation. This upsampling is a deliberate and necessary deviation from the raw CIFAR-10 format: without it, the pretrained convolutional filters – tuned to the spatial scale of ImageNet images – would not be applicable to the much smaller native CIFAR-10 resolution, and the benefit of transfer learning would be lost.

8. Experimental Procedure

The experiment was carried out using Python and PyTorch in Google Colab, with automatic detection of the available compute device (CUDA GPU or CPU) and a fixed random seed for reproducibility.

Property	**Value**
Device Used	CUDA GPU
Random Seed	42
Framework	PyTorch
Backbone	ResNet50 (`torchvision.models.resnet50`), ImageNet pretrained

Task 1: Dataset Preparation

CIFAR-10 was loaded from the Hugging Face Hub and wrapped in a custom PyTorch `Dataset` class that applies the resize-to-224×224, tensor-conversion and ImageNet-normalization transform pipeline to every image. Ten representative sample images were displayed with their true class names, and the training and testing dataset dimensions were printed.

Task 2: Transfer Learning Setup

A ResNet50 model was loaded with pretrained ImageNet weights, its convolutional base was frozen, and its original 1000-class classification layer was replaced with a new head consisting of the existing Global Average Pooling layer followed by a Dense (Linear) layer with ReLU activation and a final 10-unit output layer, with Softmax realized implicitly via the Cross-Entropy loss.

Task 3: Model Training (Frozen Backbone)

The new classification head was trained for 10 epochs using the Adam optimizer with a learning rate of 0.001, a batch size of 32 and the Cross-Entropy loss function, while every convolutional layer of the ResNet50 backbone remained frozen.

Task 4: Fine-Tuning

The last convolutional block of ResNet50 (`layer4`) was unfrozen, and the model was trained for a further 5 epochs at a reduced learning rate of 0.0001, so that the pretrained backbone weights could adapt to CIFAR-10 without being disrupted too aggressively. The classification accuracy before and after fine-tuning was then compared directly.

Task 5: Model Evaluation

The fine-tuned model was evaluated on the untouched testing set of 10,000 images. Accuracy, macro-averaged Precision, Recall and F1-score, the per-class Classification Report and the Confusion Matrix were computed, and a sample of misclassified test images was inspected directly.

9. Model Architecture (ResNet50 Transfer-Learning Head)

The transfer-learning model implemented for this experiment consists of the full ResNet50 convolutional backbone, pretrained on ImageNet, followed by a new classification head, as summarized below.

Layer	**Configuration**
Backbone	ResNet50 convolutional base (ImageNet pretrained, frozen during Task 3)
Global Average Pooling	ResNet50's existing adaptive average pooling layer
FC1	Linear(2,048 $\rightarrow$ 256), ReLU
Dropout	Dropout($p = 0.3$)
FC2 (Output)	Linear(256 $\rightarrow$ 10), Softmax (via Cross-Entropy loss)

The network was compiled with the Cross-Entropy loss function. During Task 3, only the parameters of the new classification head (FC1 and FC2) were trainable, while during Task 4 the parameters of `layer4`, the final residual block of the ResNet50 backbone, were additionally unfrozen and trained alongside the head, allowing the deepest, most task-specific convolutional filters to adapt to the visual characteristics of CIFAR-10.

10. Hyperparameter Study

Hyperparameter	**Values**
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

Of this grid, only a single configuration was actually trained in this experiment – a learning rate of 0.001 with 10 frozen-backbone epochs followed by a learning rate of 0.0001 for 5 fine-tuning epochs, a batch size of 32, the Adam optimizer, 256 dense units, and partially frozen layers (`layer4` unfrozen during fine-tuning) – and it is marked below as the reference row against which the remaining, unrun configurations are listed for completeness.

Configuration	LR	Batch	Epochs	Optim.	Dense	Frozen Layers	Test Acc.
Actually Trained (this run)	0.001	32	10	Adam	256	Partial (layer4)	90.08%
LR = 0.0001, Batch = 32	0.0001	32	10	Adam	256	Not run	–
Batch = 16	0.001	16	10	Adam	256	Not run	–
Batch = 64	0.001	64	10	Adam	256	Not run	–
Epochs = 20	0.001	32	20	Adam	256	Not run	–
Optimizer = SGD	0.001	32	10	SGD	256	Not run	–
Dense Units = 128	0.001	32	10	Adam	128	Not run	–
Frozen Layers = All	0.001	32	10	Adam	256	Not run	–

11. Parameter Calculation / Parameter Counts

The total and trainable parameter counts of the transfer-learning model were printed directly by PyTorch at each stage of the experiment.

Stage	Trainable Parameters
Total Parameters (entire model)	24,035,146
Trainable Parameters (Task 3, frozen backbone)	527,114
Frozen Parameters (Task 3, frozen backbone)	23,508,032
Trainable Parameters (Task 4, after unfreezing layer4)	15,491,850

During the frozen-backbone phase, only 527,114 of the model's 24,035,146 total parameters – roughly 2.2% – were trainable, confirming that almost the entire ResNet50 convolutional base was successfully held fixed at its pretrained ImageNet weights while only the new classification head was updated. Once `layer4` was unfrozen for fine-tuning, the trainable parameter count rose sharply to 15,491,850, close to two-thirds of the total model, reflecting the fact that ResNet50's final residual block, which contains its widest and most numerous filters, accounts for the large majority of the backbone's own parameter budget.

12. Experimental Results

12.1 Before vs After Fine-Tuning

Stage	Train Acc.	Val Acc.	Train Loss	Val Loss
Frozen Backbone (Final Epoch, 10)	84.59%	82.89%	0.4388	0.4952
Fine-Tuned (Final Epoch, 5)	98.68%	90.08%	0.0402	0.3536

Fine-tuning the last convolutional block produced a substantial improvement over the frozen-backbone baseline: validation accuracy rose from 82.89% to 90.08%, a gain of over 7 percentage points, in only 5 additional epochs, while validation loss fell from 0.4952 to 0.3536. Training accuracy also rose sharply, from 84.59% to 98.68%, and training loss fell from 0.4388 to a very low 0.0402, indicating that once `layer4` was unfrozen the model was able to fit the training data almost perfectly. The frozen-backbone phase took 2,265.73 seconds to complete 10 epochs, while fine-tuning took a further 1,449.56 seconds for 5 epochs, giving a combined total training time of 3,715.28 seconds.

12.2 Test Set Evaluation

Metric	**Value**
Test Accuracy	90.08%
Precision (macro)	90.18%
Recall (macro)	90.08%
F1-Score (macro)	90.07%

Class	**Precision**	**Recall**	**F1-Score**	**Support**
airplane	0.92	0.89	0.90	1,000
automobile	0.96	0.93	0.94	1,000
bird	0.92	0.86	0.89	1,000
cat	0.82	0.82	0.82	1,000
deer	0.89	0.89	0.89	1,000
dog	0.85	0.88	0.86	1,000
frog	0.89	0.95	0.92	1,000
horse	0.95	0.89	0.92	1,000
ship	0.89	0.97	0.93	1,000
truck	0.93	0.94	0.93	1,000
Macro/Weighted Avg	**0.90**	**0.90**	**0.90**	**10,000**

The per-class results show automobile (96% precision, 93% recall) and ship (89% precision, 97% recall) as the most reliably classified categories, alongside strong performance on truck and horse. The weakest class by a clear margin is cat, at only 82% precision and 82% recall, with dog also comparatively weak at 85% precision and 88% recall – the same cat/dog confusion pattern that is a well-known difficulty in CIFAR-10 classification, since the two classes share broadly similar body shapes, fur textures and typical backgrounds.

12.3 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time	Source
LeNet-5	60K	–	–	Literature
AlexNet	61M	–	–	Literature
VGG16	138M	–	–	Literature
GoogleNet	6.8M	–	–	Literature
ResNet50	24,035,146	90.08	3,715.28 s	Measured (this run)

Since only ResNet50 was actually trained in this experiment, it is the only row in this table with measured accuracy and training-time figures; the remaining rows reproduce the literature parameter counts from Section 5 for reference and are not directly comparable in accuracy or training time, since LeNet-5, AlexNet, VGG16 and GoogleNet were not trained on CIFAR-10 in this experiment. Notably, the transfer-learning ResNet50 model's total parameter count of 24,035,146 is close to, but slightly lower than, the 25.6M figure typically quoted for the base ResNet50 architecture, since the original 1000-class ImageNet classification layer (which contributes just over 2 million parameters) was removed and replaced with the smaller 256-unit head used in this experiment.

13. Plots and Interpretation

13.1 Sample Images

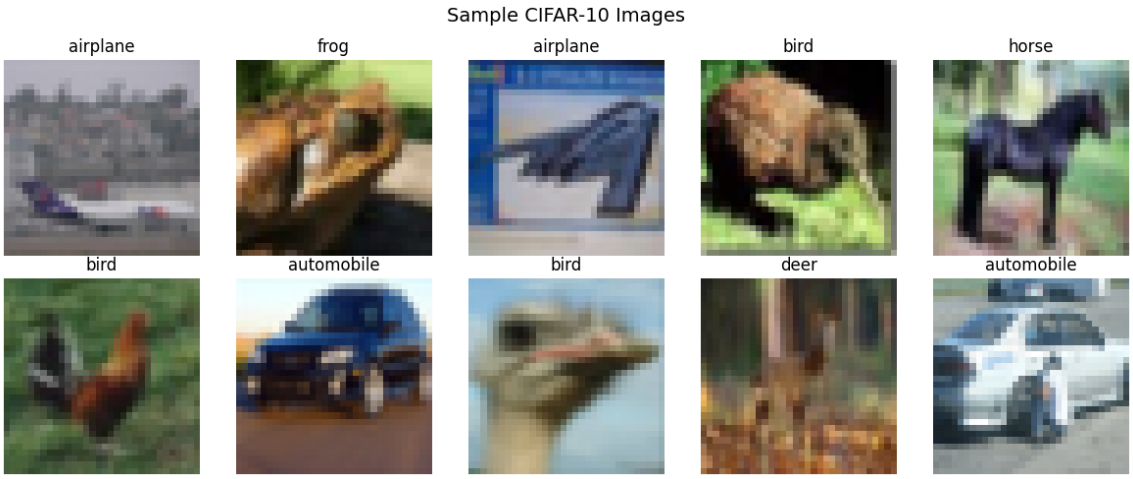


Figure 1: Ten Sample Images from the CIFAR-10 Dataset with True Class Labels

The sample images span both vehicles (airplane, automobile) and animals (frog, bird, horse, deer), each shown at low native resolution against a variety of natural backgrounds. Several images, such as the frog and the ostrich labelled "bird", show that even at 32×32 resolution, distinguishing certain animal classes visually is genuinely difficult, foreshadowing some of the confusion later observed between visually related animal categories in the confusion matrix.

13.2 Training Accuracy (Frozen Backbone)

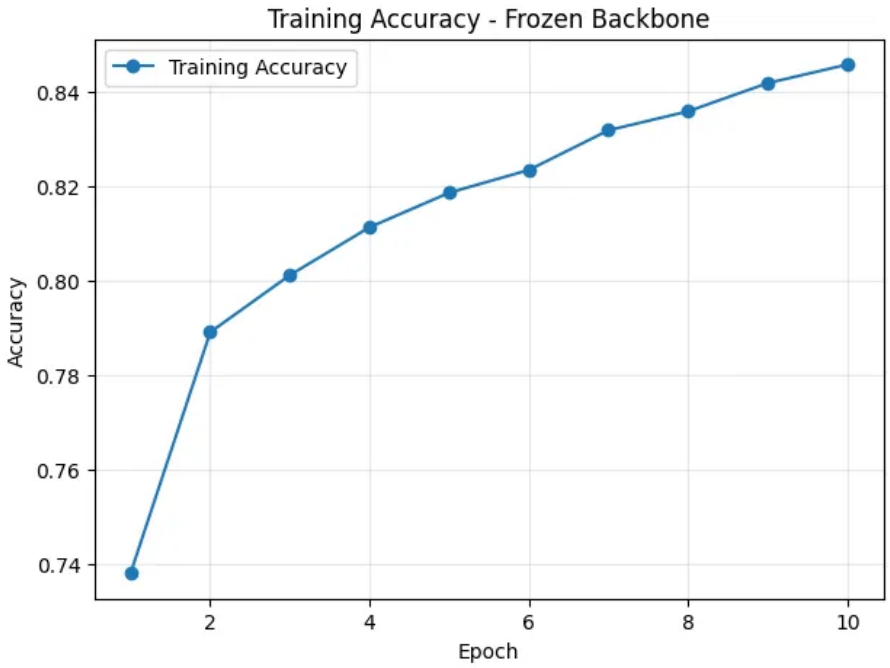


Figure 2: Training Accuracy vs Epoch, Frozen Backbone

Training accuracy rises steadily and smoothly across all 10 frozen-backbone epochs, from 73.80% in the first epoch to 84.59% by the tenth, with the steepest gains occurring in the first two epochs before the curve begins to flatten. This smooth, well-behaved rise, without the erratic jumps sometimes seen when training an entire network from scratch, reflects the fact that only the small classification head is being optimized here, on top of already-powerful pretrained features.

13.3 Validation Accuracy (Frozen Backbone)

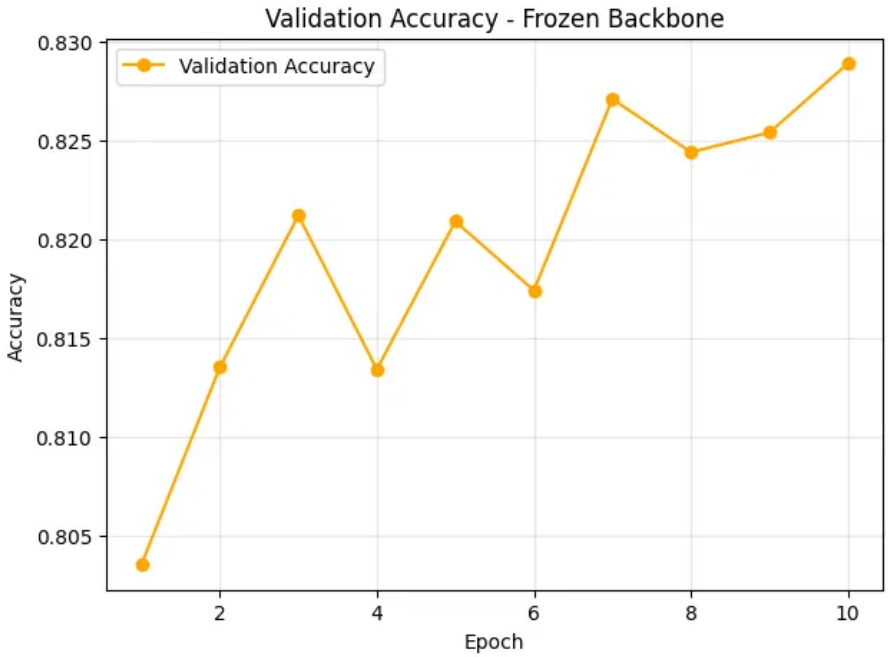


Figure 3: Validation Accuracy vs Epoch, Frozen Backbone

Validation accuracy climbs from 80.35% to 82.89% over the 10 frozen-backbone epochs, with some mild non-monotonic fluctuation around epochs 4 and 6 rather than a perfectly smooth increase. Unlike the overfitting pattern observed in Experiment 3, validation accuracy here tracks training accuracy fairly closely throughout, and does not decline in later epochs, indicating that the frozen ResNet50 backbone provides features general enough that the small trainable head does not overfit within this training budget.

13.4 Training Loss (Frozen Backbone)

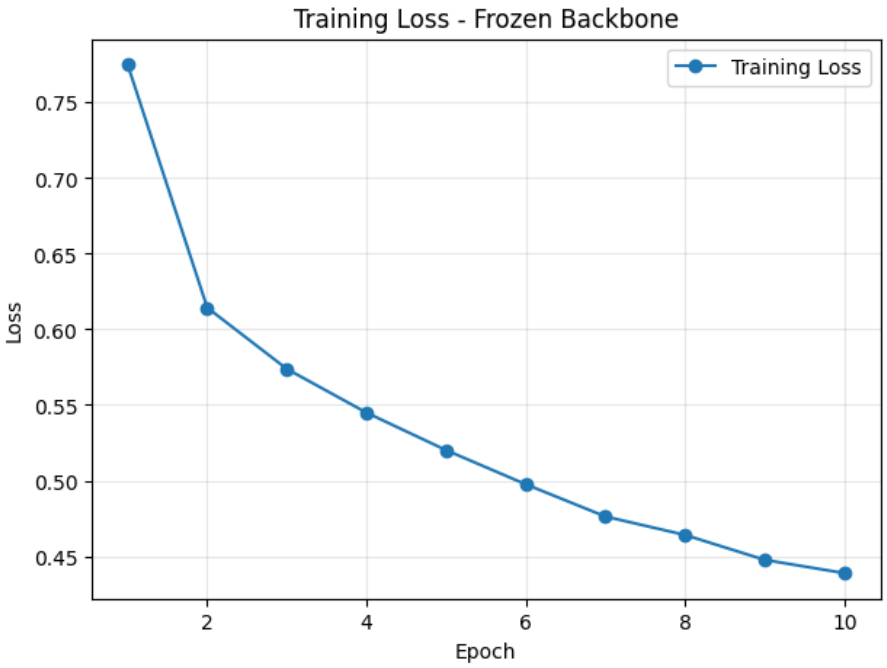


Figure 4: Training Loss vs Epoch, Frozen Backbone

Training loss falls sharply from 0.7751 in the first epoch to 0.6143 in the second, before continuing to decrease more gradually to 0.4388 by the tenth epoch. The steep initial drop is typical of transfer learning, since the randomly initialized classification head starts far from a good solution while the frozen backbone already supplies strong, discriminative features, so the head converges quickly once training begins.

13.5 Validation Loss (Frozen Backbone)

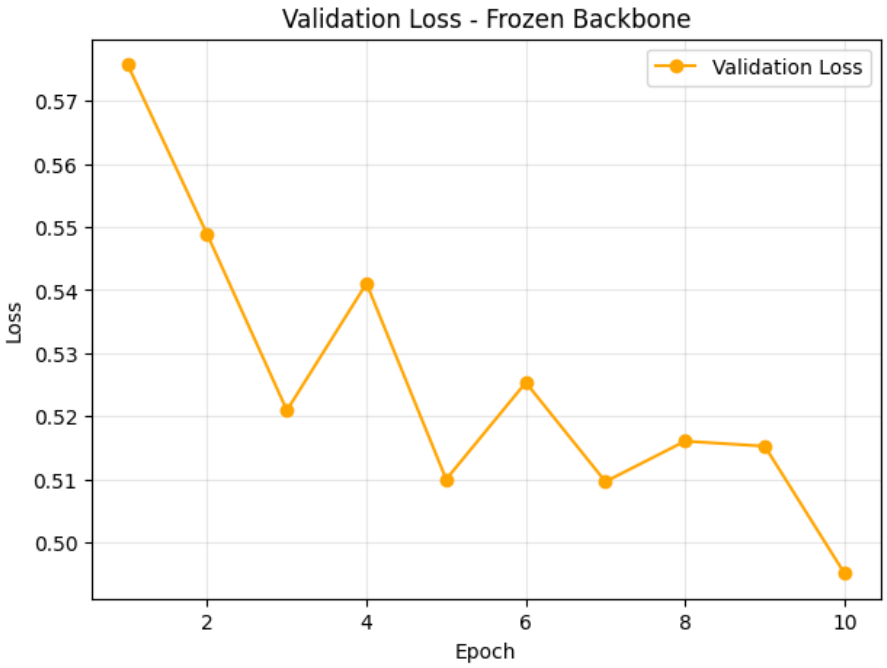


Figure 5: Validation Loss vs Epoch, Frozen Backbone

Validation loss falls from 0.5757 to a low of 0.4952 by the tenth epoch, but the curve is visibly noisier than the training loss curve, rising briefly around epochs 4 and 6 before falling again. This mild fluctuation, without any sustained upward trend, is consistent with the validation accuracy behaviour in Section 13.3 and further supports the conclusion that the frozen-backbone model is not yet overfitting within these 10 epochs.

13.6 Confusion Matrix

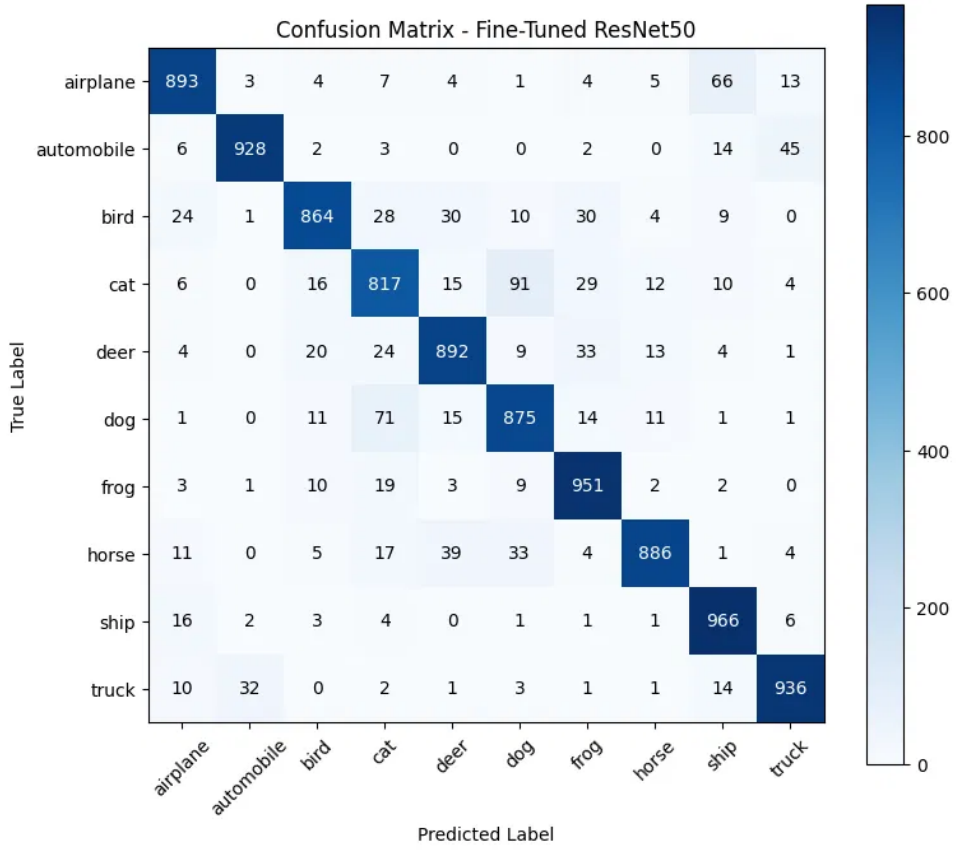


Figure 6: Confusion Matrix of the Fine-Tuned ResNet50 on the CIFAR-10 Testing Set

The confusion matrix shows a strong, dominant diagonal across all ten classes, consistent with the 90.08% overall test accuracy. The most prominent off-diagonal confusions are airplane predicted as ship (66 instances) and truck predicted as automobile (32 instances), both plausible given that airplanes and ships are frequently photographed against similar plain sky or water backgrounds, and automobiles and trucks share a broadly similar boxy silhouette. Cat and dog again show mutual confusion (91 dogs predicted as cat, 71 cats predicted as dog), the same pairing flagged as the model's weakest classes in Section 12.2, while classes such as ship, frog and automobile show very few off-diagonal entries, confirming their high precision and recall.

13.7 Misclassified Images



Figure 7: Eight Misclassified Test Images with True and Predicted Labels

Several of the misclassified examples are genuinely ambiguous even on visual inspection: a black-and-white dog with a patterned background is confused with cat, and a bird photographed against dense, textured foliage is confused with frog, both plausible errors given the low resolution and close visual similarity between these class pairs. The misclassified truck, photographed at an unusual angle against a light background, is predicted as automobile, again consistent with the vehicle confusion identified in the confusion matrix. These examples suggest that a meaningful share of the model's remaining errors stem from genuine visual ambiguity in the underlying images rather than from a systematic weakness in the learned representation itself.

14. Result Analysis

The single most informative comparison in this experiment is between the frozen-backbone baseline and the fine-tuned model: unfreezing just the last residual block of ResNet50 and training it for only 5 further epochs at a reduced learning rate raised validation accuracy from 82.89% to 90.08%, a gain of more than 7 percentage points, at the cost of roughly 1,450 additional seconds of training time – a considerably better return on training time than the 10 epochs spent training the frozen-backbone head alone. Unlike the CNN trained entirely from scratch in Experiment 3, which showed a pronounced 28.8-percentage-point gap between training and testing accuracy indicative of severe overfitting, the fine-tuned ResNet50 model here shows a smaller, though still present, gap of roughly 8.6 percentage points between its final training accuracy (98.68%) and its test accuracy (90.08%), suggesting that while some overfitting does occur once layer4 is unfrozen and its 15-million-plus parameters become trainable, the pretrained ImageNet features occupied by the remainder of the frozen backbone provide considerably more regularization than a randomly initialized network of comparable depth would. The macro-averaged Precision, Recall and F1-score of 90.18%, 90.08% and 90.07% respectively are all very close to one another and to the overall accuracy, indicating that performance is fairly balanced across the ten classes rather than being propped up by a small number of especially easy categories, though the cat and dog classes remain the model's clear weak point, mirroring a well-documented difficulty in CIFAR-10 classification generally.

15. Discussion

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet won the 2012 ImageNet Challenge by a wide margin, proving deep CNNs could outperform traditional computer vision methods at scale. Its use of ReLU activations avoided the vanishing gradients that plagued Sigmoid/Tanh networks, Dropout controlled overfitting in its large fully connected layers, and GPU training made a 61-million-parameter network computationally practical, together reigniting mainstream interest in deep learning.

2. Why does VGG16 use only 3×3 convolution filters?

Stacking multiple 3×3 filters achieves the same effective receptive field as a single larger filter (two 3×3 layers match a 5×5 receptive field) while using fewer parameters and adding an extra non-linearity between them. This uniform, simple design lets VGG16 go deeper than earlier architectures while keeping the network easy to reason about and extend layer by layer.

3. Explain the advantages of the Inception module.

The Inception module applies 1×1, 3×3 and 5×5 convolutions and pooling in parallel, then concatenates their outputs, capturing multi-scale features at every stage rather than committing to one filter size. Its 1×1 convolutions reduce channel depth before expensive larger filters, keeping GoogleNet's parameter count (6.8M) far below AlexNet or VGG16 despite comparable or better accuracy.

4. What is the purpose of residual learning?

Residual learning reformulates a block's target from the full mapping $H(x)$ to the residual $F(x) = H(x) - x$, added back to the input via a skip connection. This gives gradients a direct path backward through the network regardless of depth, solving the vanishing-gradient problem that previously made networks beyond roughly 20 layers difficult to train.

5. Differentiate LeNet and ResNet.

LeNet-5 (1998) is a shallow, 7-layer network with only around 60 thousand parameters, using average pooling and no skip connections, designed for simple digit recognition. ResNet (2015) reaches 50 or more layers using residual skip connections that avoid vanishing gradients, achieving far higher accuracy on complex, natural images at the cost of dramatically greater parameter count and computation.

6. What is Transfer Learning?

Transfer learning reuses a CNN already pretrained on a large dataset such as ImageNet for a new, typically smaller, target task, rather than training from randomly initialized weights. The pretrained convolutional base, which already encodes general-purpose visual features, is frozen and combined with a new classification head suited to the target classes, dramatically reducing the data and compute needed to reach strong accuracy.

7. Why is fine tuning required?

Training only the new classification head on top of a frozen backbone limits the model to features learned from ImageNet, which may not perfectly match the target dataset's visual characteristics. Fine tuning unfreezes the deepest, most task-specific convolutional layers and

retrains them at a small learning rate, letting the pretrained features adapt to the new domain, which in this experiment raised validation accuracy from 82.89% to 90.08%.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution enlarges a kernel's receptive field by inserting gaps between its weights, without changing the input's spatial size or the parameter count, and is used to capture broader context in tasks like segmentation. Transpose convolution instead performs learnable upsampling, increasing a feature map's spatial dimensions, and is used in decoders, GANs and super-resolution where a compact representation must be projected back to full resolution.

9. Why do pretrained models converge faster?

A pretrained model's convolutional weights already encode general-purpose features – edges, textures, shapes – learned from millions of ImageNet images, so training on a new task only needs to adjust or add a much smaller set of task-specific parameters. This was directly visible in this experiment, where the frozen-backbone head's training loss fell from 0.7751 to 0.6143 within the very first two epochs.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 requires only around 60 thousand parameters and correspondingly little computation, since it is shallow and operates on small, low-resolution inputs, making it fast to train even on modest hardware. ResNet50, with roughly 25.6 million parameters spread across 50 layers, requires substantially more computation per forward and backward pass; in this experiment, even with a frozen backbone, a single epoch on 50,000 224×224 images took several minutes on a GPU.

16. Additional Exercise Solutions

Exercise 1: Implement transfer learning using VGG16.

This experiment implemented transfer learning using ResNet50 rather than VGG16, since ResNet50 was selected as the backbone for this run from the four options offered by the lab manual, pairing directly with the ResNet theory covered in Section 4.5. The same overall workflow described in Section 6 – loading pretrained ImageNet weights, removing the original classifier, freezing the convolutional base, adding a Global Average Pooling layer followed by a Dense layer with ReLU activation and a 10-unit Softmax output, then fine-tuning the last convolutional block – would apply essentially unchanged if VGG16 were substituted in its place, using `torchvision.models.vgg16` instead of `resnet50`. The main practical difference would be VGG16's substantially larger parameter count (138M versus ResNet50's 25.6M), driven mainly by its own large, uniform 3×3 convolutional stack and fully connected layers, which would likely increase both training time per epoch and GPU memory requirements relative to the ResNet50 run actually performed here.

Exercise 2: Repeat the experiment using ResNet50.

ResNet50 was the backbone actually used for the transfer-learning run in this experiment, as described throughout Sections 8 through 12. The frozen-backbone head was trained for 10 epochs using the Adam optimizer at a learning rate of 0.001, reaching a final training accuracy of 84.59% and a validation accuracy of 82.89%, before the last convolutional block (`layer4`) was unfrozen and fine-tuned for a further 5 epochs at a reduced learning rate of 0.0001, raising

validation accuracy to 90.08% and yielding a final test accuracy, on the fully held-out testing set, of 90.08% with a macro F1-score of 90.07%. This single run therefore already directly answers this exercise, and the results discussed in Sections 12 and 14 constitute the complete outcome of repeating the experiment with ResNet50 as the chosen architecture.

Exercise 3: Compare Adam and SGD optimizers.

Only the Adam optimizer was actually used for training in this experiment, as specified in the lab manual's Task 3 configuration and reflected in the hyperparameter study table of Section 10, where the Optimizer = SGD row is explicitly marked as not run. Adam combines per-parameter adaptive learning rates with momentum, typically converging faster than plain SGD in the early epochs of training, which is consistent with the sharp initial drop in training loss (from 0.7751 to 0.6143) observed within the first two frozen-backbone epochs in Section 13.4. SGD, particularly with momentum, can sometimes generalize slightly better over long training runs, but usually requires more careful learning-rate tuning and more epochs to reach comparable accuracy, an experiment that was outside the scope of the single configuration actually trained here.

Exercise 4: Compare training with frozen layers and fine tuning.

This comparison is directly available from Sections 8, 12.1 and 14 of this report: with the backbone fully frozen, the model reached a validation accuracy of 82.89% and a validation loss of 0.4952 after 10 epochs, whereas fine-tuning `layer4` for 5 additional epochs raised validation accuracy to 90.08% and reduced validation loss to 0.3536. This represents a clear, substantial improvement from fine-tuning relative to training the frozen-backbone head alone, though it comes with an increase in the gap between training and testing accuracy (from roughly 1.7 percentage points in the frozen phase to roughly 8.6 percentage points after fine-tuning), reflecting the trade-off between the greater representational flexibility unlocked by fine-tuning and its correspondingly higher risk of overfitting.

Exercise 5: Evaluate the model using Precision, Recall and F1-score.

The fine-tuned ResNet50 model was evaluated on the full 10,000-image CIFAR-10 testing set in Section 12.2, achieving a macro-averaged Precision of 90.18%, Recall of 90.08% and F1-score of 90.07%, figures that are all very close to the overall accuracy of 90.08%, indicating fairly balanced performance across all ten classes rather than a result skewed by a small number of easy categories. The per-class classification report shows this balance is not perfectly uniform, however: automobile and ship both exceed 93% F1-score, while cat lags well behind at only 82% F1-score, so the macro average, while broadly representative, still masks a meaningful spread in per-class difficulty.

Exercise 6: Compare the performance of LeNet, AlexNet and ResNet.

Only ResNet50 was actually trained and evaluated in this experiment, reaching a measured test accuracy of 90.08% as reported in Section 12.3; LeNet-5 and AlexNet were not trained on CIFAR-10 here, so no directly comparable measured accuracy figures exist for them in this run. Based on the architectural theory covered in Sections 4.1, 4.2 and 4.5, however, LeNet-5's shallow 7-layer design and roughly 60 thousand parameters would be expected to struggle considerably on a natural-image dataset like CIFAR-10 relative to ResNet50's 50-layer, residual-connection architecture, while AlexNet, despite its much larger 61-million parameter count, lacks ResNet's skip connections and would likely be harder to train effectively at comparable depth, making the transfer-learning ResNet50 approach used here the clear practical choice among the three for this dataset.

17. Conclusion

A transfer-learning pipeline was built in PyTorch around a pretrained ResNet50 backbone and applied to the CIFAR-10 dataset, loaded from the Hugging Face Hub and resized to 224×224 to match the backbone's expected input resolution. The convolutional base was first frozen while a new classification head, consisting of Global Average Pooling followed by a 256-unit Dense layer with ReLU activation and a 10-unit output layer, was trained for 10 epochs using the Adam optimizer, reaching a validation accuracy of 82.89%; the last residual block of the backbone was then unfrozen and fine-tuned for 5 further epochs at a reduced learning rate of 0.0001, raising validation accuracy to 90.08% and yielding a final test accuracy of 90.08%, with macro-averaged Precision, Recall and F1-score of 90.18%, 90.08% and 90.07% respectively. This fine-tuning stage, though comprising only 5 of the 15 total training epochs, contributed the majority of the improvement in generalization performance relative to the frozen-backbone baseline, confirming that adapting even a small, well-chosen subset of a pretrained network's deepest layers can be substantially more valuable than training a small classification head alone. The confusion matrix and misclassified-image inspection identified cat and dog as the model's most persistent source of error, consistent with the visual similarity between these two categories noted throughout the computer vision literature, while classes such as ship, automobile and frog were classified with high reliability. Overall, the experiment reinforced practical understanding of the evolution of CNN architectures, the mechanics and benefits of transfer learning, and the measurable trade-off between the reduced overfitting risk of a frozen backbone and the higher achievable accuracy unlocked by careful fine-tuning.

18. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio, Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan, Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation.
   https://www.tensorflow.org
8. Keras Documentation.
   https://keras.io
9. PyTorch Documentation.
   https://pytorch.org/docs/stable/index.html
10. TorchVision Models Documentation.
    https://pytorch.org/vision/stable/models.html
11. Hugging Face Datasets Documentation.
    https://huggingface.co/docs/datasets

19.Source Code

The complete implementation was carried out in Python using Google Colab.

https://github.com/K-Mithran/Deep-Learning/blob/main/Lab4/DL_4.ipynb